
Fibonacci in Littleman

A picture-first guide for a complete beginner

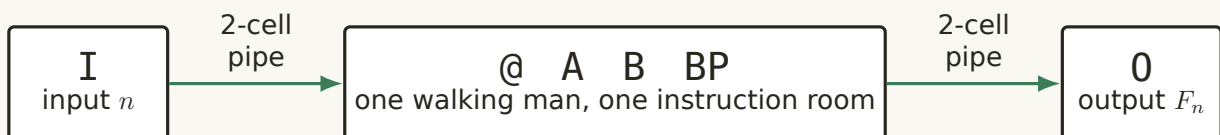
The entire idea is this:

$$\mathbf{A} = F_k$$

$$\mathbf{B} = F_{k+1}$$

$$\mathbf{BP} = n - k$$

One lap changes k into $k + 1$. After n laps, $\mathbf{A} = F_n$, so the man sends $\mathbf{A}$ to the output.



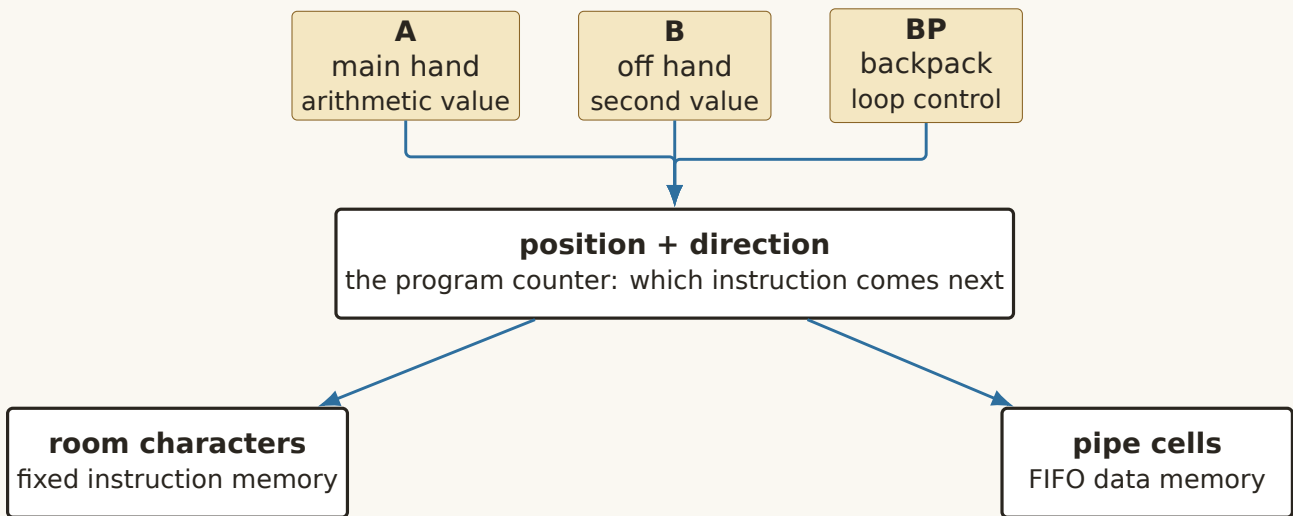
What this guide assumes: nothing beyond ordinary addition. It explains what the rooms are, what every used character means, where the man walks, what is in each register after every important operation, why the loop stops, and why the answer is correct.

Contents

1 First: what kind of “computer” is this?	2
1.1 One tick	2
2 The problem	2
3 The complete program	3
3.1 The same program as a map	4
4 Every character used	5
5 Phase A: initialization, painfully slowly	5
6 Phase B: the loop controller	6
7 One lap: why + W advances Fibonacci	6
8 Full worked example: input $n = 5$	7
8.1 Initialization	7
8.2 Five laps	7
9 Where the time goes	8
10 Why there is no relay room here	9
11 Common wrong versions	9
12 How to watch it yourself	10
12.1 In the archived web editor	10
12.2 With the repository simulator	10
13 Validation evidence	10
14 Your compact program: before and after	12
14.1 The measured result	12
15 Exactly why the optimized program is faster	13
15.1 Optimization 1: prepare A and B in four operations	13
15.2 Optimization 2: fold the control path around the arithmetic	13
15.3 Optimization 3: remove walking-only cells from every lap	14
15.4 Where the two formulas come from	14

1 First: what kind of “computer” is this?

Do not imagine that the little man must remember the entire computation in his head. Imagine a tiny CPU walking over its own program:



The hands are working registers, not total memory. A normal CPU can process a large file with only a few registers because it handles one piece at a time and stores the rest elsewhere. Littleman works the same way. In this Fibonacci program there is no large data set: two consecutive numbers are sufficient, so **A** and **B** are all the data memory we need.

1.1 One tick

Time advances in discrete ticks. For our program, the useful mental model is:

1. Values in pipes move one cell toward their destination.
2. The man executes the character under his feet.
3. Unless blocked or halted, he moves one cell in his current direction.

He begins on @ facing right. A blank space is a real instruction: “do nothing, then keep walking.”

2 The problem

We choose the standard Fibonacci convention:

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \quad (n \geq 2).$$

The machine receives one integer n , computes F_n , emits that one integer, and halts. We restrict $0 \leq n \leq 92$, because F_{92} is the largest Fibonacci answer that fits in a positive signed 64-bit integer.

n	0	1	2	3	4	5	6	7	8	9	10
F_n	0	1	1	2	3	5	8	13	21	34	55

Why this is a perfect first example. Fibonacci needs a loop, arithmetic, input, output, and all three registers, but it does *not* need a pipe ring or a second working man. We can see the core execution model without extra machinery.

3 The complete program

Copyable .man source

```

++ +-----+
|I|>>|@rb0M1Wv|
++ +-----+
      > dsH|>>|0|
      +     +-+
      W     +-+
      m
      ^ <
+-----+

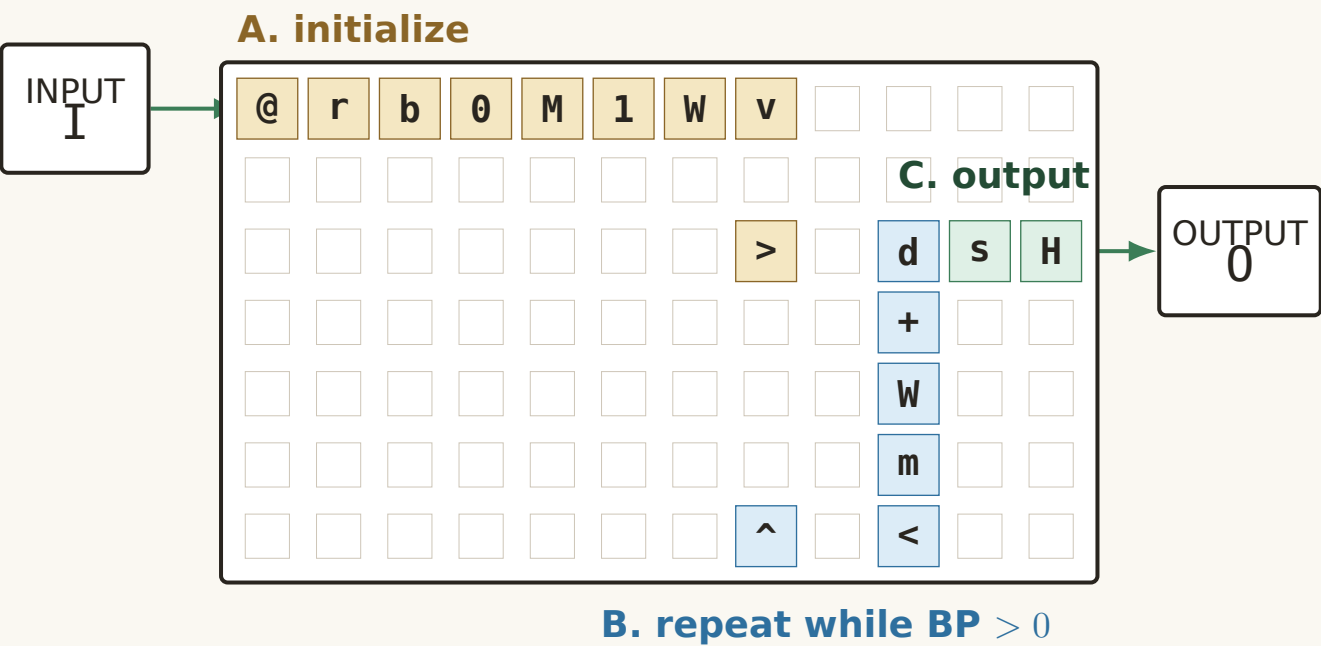
```

The parser sees three rooms:

- `|I|`: the special input room; it contains no man.
- the large middle rectangle: the only instruction room; it contains one `@`, hence one man.
- `|0|`: the special output room; it contains no man.

The `>>` pairs are two-cell pipes. The left pipe carries n into the worker. The right pipe carries the final answer out.

3.1 The same program as a map



Read the colors in order: gold runs once; blue is the repeated lap; green runs once at the end. The arrows show the man's route, not the movement of numbers in a pipe.

4 Every character used

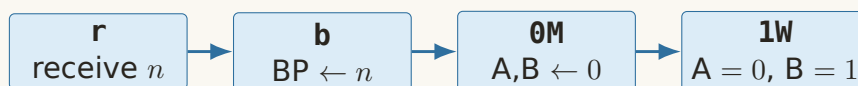
Cell	Name	Exact effect when executed
@	spawn	The man starts here facing right. Later it behaves like a blank cell.
r	receive	Take one value from the nearest incoming pipe and put it in A . Wait here if no value has arrived.
b	backpack write	Copy A into BP . A is unchanged.
0, 1	constants	Replace A with that digit.
M	copy hand	Copy A into B . A is unchanged.
W	swap hands	Exchange A and B .
+	add	Set $\mathbf{A} \leftarrow \mathbf{A} + \mathbf{B}$. B is unchanged.
m	decrement BP	Set $\mathbf{BP} \leftarrow \mathbf{BP} - 1$.
d	BP branch	If $\mathbf{BP} > 0$, turn clockwise. Otherwise keep the current direction. BP itself is unchanged.
>, <, ^, v	direction	Face right, left, up, or down.
s	send	Put A into the nearest outgoing pipe. Wait here if the pipe's first cell is occupied.
H	halt	Stop this man permanently.
blank	no-op	Do nothing and continue in the same direction.

Easy misconception: the backpack is not a normal third hand. There is no instruction “copy BP back into A.” In this program we never need its numerical value for arithmetic. We only write it, decrement it, and ask *is it still positive?*

5 Phase A: initialization, painfully slowly

Input is n . Initially every register is zero:

$$\mathbf{A} = 0, \quad \mathbf{B} = 0, \quad \mathbf{BP} = 0.$$

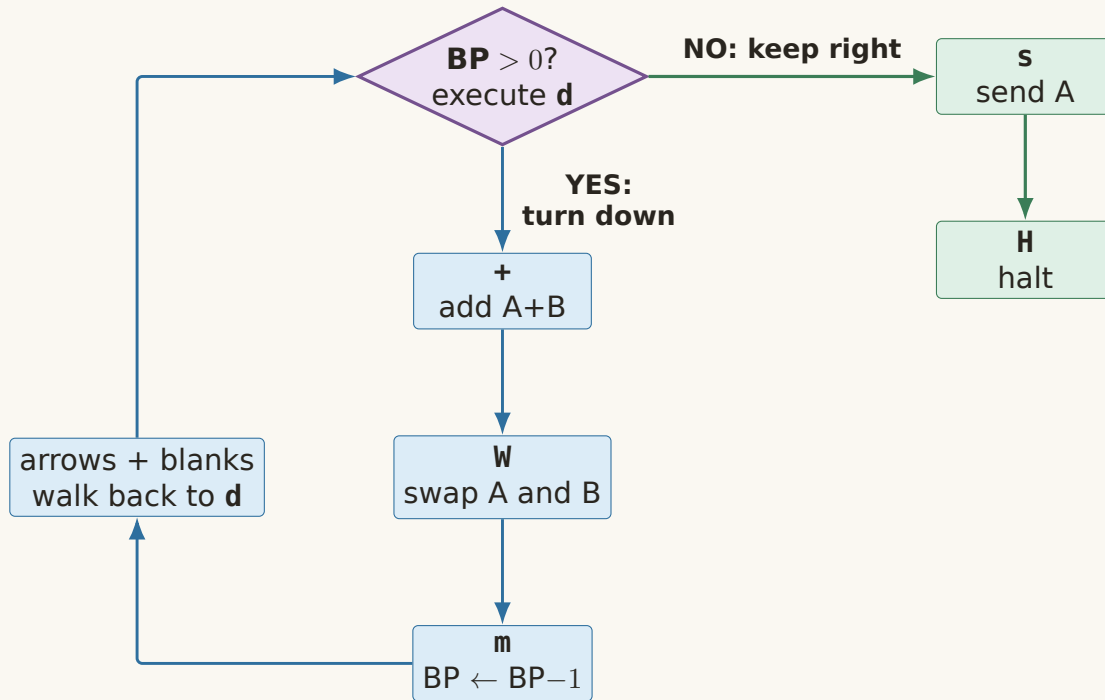


Just executed	A	B	BP	Why
nothing	0	0	0	All registers start at zero.
r	n	0	0	Input arrives in the main hand.
b	n	0	n	Save the loop count in BP.
0	0	0	n	Start constructing the Fibonacci pair.
M	0	0	n	Copy 0 into B.
1	1	0	n	Temporarily put 1 in A.
W	0	1	n	Swap: now $\mathbf{A} = F_0$, $\mathbf{B} = F_1$.

Why the apparently silly sequence 0 M 1 W? Constants only load **A**; there is no “put 1 directly in B.” So we first make $B = 0$, then load 1 into A, then swap. The result is the exact starting pair $(0, 1)$.

6 Phase B: the loop controller

After initialization, **v** turns the man downward. Two rows later, **>** turns him right. He arrives at **d** facing right.



At d, direction is part of the condition. The instruction does not contain an address or a label. It merely turns:

- facing right + clockwise turn = face down, entering the blue loop;
- facing right + no turn = keep walking right, reaching **s H**.

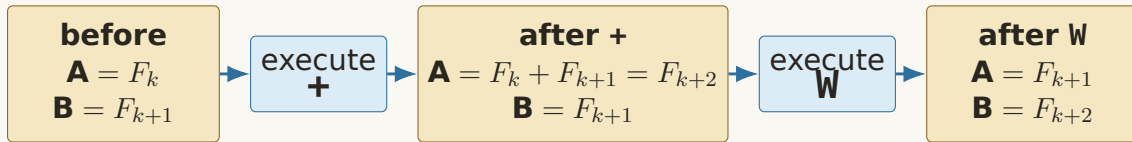
That is geometric control flow. The man’s heading replaces an ordinary machine-code branch target.

Why test BP before performing the first addition? Because n can be zero. If $BP = 0$, the machine must output $F_0 = 0$ immediately. Entering the loop once would incorrectly output $F_1 = 1$.

7 One lap: why + W advances Fibonacci

Assume the hands contain two neighboring Fibonacci numbers:

$$\mathbf{A} = F_k, \quad \mathbf{B} = F_{k+1}.$$



Before the lap we hold (F_k, F_{k+1}) . After **+** **W** we hold (F_{k+1}, F_{k+2}) . That is the same shape with the index increased by one. Then **m** decreases the remaining-lap count by one.

This repeating statement is the whole proof:

$$(\mathbf{A}, \mathbf{B}, \mathbf{BP}) = (F_k, F_{k+1}, n - k)$$

It is true before the first lap with $k = 0$. One lap makes it true for $k + 1$. When BP reaches zero, $n - k = 0$, therefore $k = n$, therefore $\mathbf{A} = F_n$.

8 Full worked example: input $n = 5$

The expected answer is $F_5 = 5$.

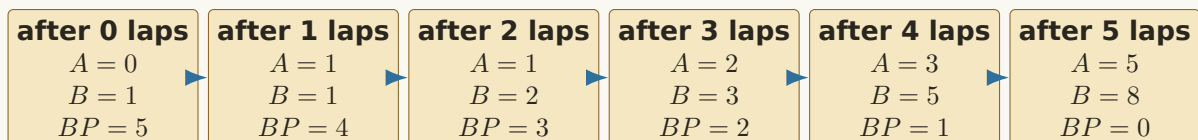
8.1 Initialization

$$(\mathbf{A}, \mathbf{B}, \mathbf{BP}) = (0, 1, 5).$$

8.2 Five laps

At d	BP positive?	after +	after W	after m
$A = 0, B = 1, BP = 5$	yes	$A = 1, B = 1$	$A = 1, B = 1$	$BP = 4$
$A = 1, B = 1, BP = 4$	yes	$A = 2, B = 1$	$A = 1, B = 2$	$BP = 3$
$A = 1, B = 2, BP = 3$	yes	$A = 3, B = 2$	$A = 2, B = 3$	$BP = 2$
$A = 2, B = 3, BP = 2$	yes	$A = 5, B = 3$	$A = 3, B = 5$	$BP = 1$
$A = 3, B = 5, BP = 1$	yes	$A = 8, B = 5$	$A = 5, B = 8$	$BP = 0$
$A = 5, B = 8, BP = 0$	no	walk right to s ; send $A = 5$		

Each arrow is one lap: execute **+** **W** **m**.



Notice the “extra” 8 in **B**. It is not a mistake. After five laps the pair is $(F_5, F_6) = (5, 8)$. The problem asks for F_5 , so the machine correctly sends **A**, not **B**.

9 Where the time goes

For the validated machine:

$$\text{ticks}(n) = 14 + 12n.$$

The constant 14 covers input arrival, initialization, the final BP test, sending, and output-pipe travel. Every additional Fibonacci lap costs 12 ticks:

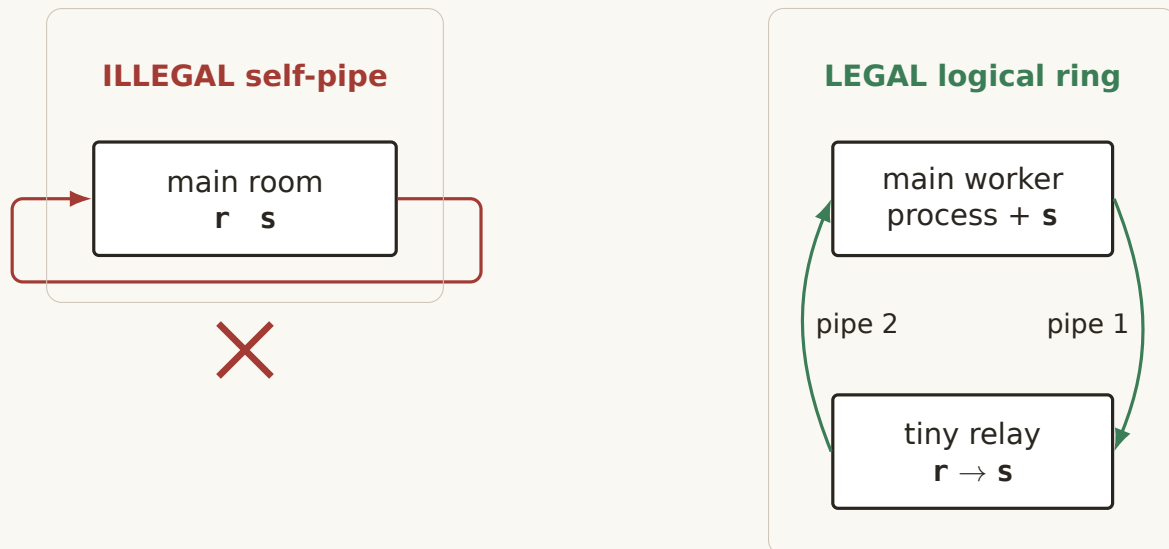
Part of one lap	ticks
Execute d , + , W , m	4
Execute the turn cells < , ^ , >	3
Walk over five blank cells	5
Total	12

Blank corridors cost time. They are not visually exciting, but the man must execute each blank as a no-op and move across it.

10 Why there is no relay room here

Your observation about relay rooms is correct:

A legal pipe must end at a room different from the room where it began. Therefore one room cannot send a pipe directly back into itself.



The relay man receives a value from pipe 1 and sends it into pipe 2. Together, the two legal pipes behave like one feedback FIFO.

Fibonacci does not need that memory. At any moment, the future depends only on the two most recent values. Those fit in **A** and **B**, so a relay would only make the program larger and slower.

11 Common wrong versions

1. Starting with **A=1, B=1**.

Then zero laps outputs 1, so $n = 0$ is wrong. Start with $(0, 1)$.

2. Doing the body before testing **BP**.

Again, $n = 0$ incorrectly performs one lap. Test first.

3. Using only **+**, without **W**.

After addition the registers are (F_{k+2}, F_{k+1}) , in the wrong order for the next addition. Swap restores the invariant.

4. Sending **B**.

After n laps, **B** is F_{n+1} . The requested answer is in **A**.

5. Trying to read **BP** into **A**.

No such instruction exists. Arrange the algorithm so **BP** is used only for control.

6. Forgetting that arrows consume ticks.

Direction cells and blank corridors are part of the running time.

7. Letting the man walk into a wall.

That is a program error, not an implicit halt. Use **H**.

12 How to watch it yourself

12.1 In the archived web editor

After the accompanying site change is published:

<https://tarstars.github.io/problems/fibonacci/editor/>

Paste the program, enter an input such as 5, put 5 in the expected output box, choose a slow speed, and use **next** one tick at a time. Watch these three things:

1. the man blocks briefly on **r** until input reaches him;
2. **A/B** become 0/1, then walk through neighboring Fibonacci pairs;
3. at $BP = 0$, **d** no longer turns the man downward.

12.2 With the repository simulator

```
uv run python - <<'PY'
import json
from pathlib import Path
from littleman.judge import judge_problem

program = Path("docs/littleman_fibonacci.man").read_text()
problem = {
    "slug": "fibonacci",
    "scoring": "footprint-tick",
    "tickCap": 5000,
    "publicTestData": [
        {"name": "five", "in": ["5"], "out": ["5"]},
    ],
}
print(judge_problem(program, problem))
PY
```

13 Validation evidence

The exact checked-in .man file used by this document passed:

n	expected/observed F_n	ticks
0	0	14
1	1	26
2	1	38
5	5	74
10	55	134
20	6,765	254
50	12,586,269,025	614
92	7,540,113,804,746,346,429	1,118

Its occupied bounding box is 24×9 , so its contest footprint would be $24^2 = 576$. This is a teaching layout, not a size-optimized contest entry. The generous room makes the loop visually obvious.

14 Your compact program: before and after

Now that the roomy teaching version is understandable, we can look at your actual contest-sized versions. They compute the same function with the same three-register invariant. The difference is how tightly the man's useful work is arranged inside the worker room.

Your original program

Correct, but not yet optimized

```
+-----+
|@rb1M0v|
| v   <<|
|>dsHW |
|>m+^  |
+-----+
^<+--+v
|I||0|<
+--+--+
```

Your optimized program

Same algorithm, shorter walking route

```
+-----+
|@rb1Wv |
|  Hsa<<|
|    m W|
|    >+^|
+-----+
^<+--+v
|I||0|<
+--+--+
```

The short pipes are legal, but they are not one-cell pipes. Each pipe contains two pipe characters between different rooms. The rule forbids a pipe from returning to its own starting room; neither program does that.

14.1 The measured result

The validation corpus was $n \in \{0, 1, 2, 5, 10, 20, 50, 92\}$. Both programs produced the expected Fibonacci number in every case.

Version	Bounding box	Tick formula	Average ticks	Score
Original	9×9	$17 + 12n$	287	23,247
Optimized	9×9	$10 + 8n$	190	15,390

The footprint stays $9^2 = 81$, but the score falls by

$$\frac{23\,247 - 15\,390}{23\,247} \times 100\% = \boxed{33.80\%}.$$

This improvement comes entirely from optimizing the program *inside* the worker room. Outer component packing cannot reduce this particular pair any further unless it also changes the 9×9 bounding square.

15 Exactly why the optimized program is faster

15.1 Optimization 1: prepare $(A, B) = (0, 1)$ in four operations

All registers begin at zero. Your original setup is:

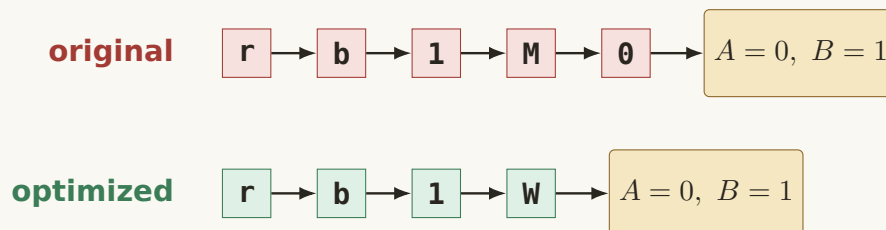
r b 1 M 0 .
five operations

after	A	B	BP	meaning
r	n	0	0	receive the input
b	n	0	n	save the loop count
1	1	0	n	put 1 in A
M	1	1	n	copy A to B
0	0	1	n	put 0 in A

The optimized setup is:

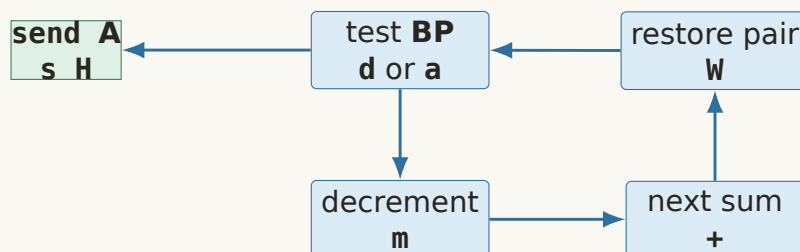
r b 1 W .
four operations

After **1**, the state is $(A, B, BP) = (1, 0, n)$. The initial zero already waiting in B is useful: **W** swaps the two hands and immediately gives $(0, 1, n)$. Thus **W** replaces the two-operation pair **M0**.



15.2 Optimization 2: fold the control path around the arithmetic

Both versions repeat the same useful work:



The two arrows leaving the test have no text drawn over them, so read this separate legend: when **BP** > 0 , the man turns into the loop; when **BP** $= 0$, he walks straight to **s** and **H**.

In the original room, **d** is approached while walking right. A positive BP makes **d** turn clockwise, so the man turns downward. In the optimized room, **a** is approached while walking left. A positive BP makes **a** turn counter-clockwise; from a leftward heading, that also means downward. For BP $= 0$, both instructions continue straight toward the send cell.

15.3 Optimization 3: remove walking-only cells from every lap

Here is one complete loop, written in the exact order in which the man executes the cells:

Version	one positive-BP lap	ticks
Original	d,>,m,+,^,W,<,blank,blank,blank,v,>	12
Optimized	a,m,>,+,^,W,<,<	8

Cells executed per lap	Original	Optimized	saved
Useful operations: branch, m, +, W	4	4	0
Direction cells	5	4	1
Blank cells	3	0	3
Total	12	8	4

The arithmetic itself is not cheaper. The man still must test BP, decrement BP, add the two Fibonacci values, and swap them. The gain comes from arranging those operations so that the return journey is shorter and contains no blank cells.

15.4 Where the two formulas come from

Part	Original	Optimized
Register preparation	5	4
Walk from preparation to the branch	8	2
Final branch, send, halt, and output travel	4	4
Fixed cost, even for $n = 0$	17	10
Cost of each Fibonacci lap	12	8

Therefore:

$$T_{\text{original}}(n) = 17 + 12n, \quad T_{\text{optimized}}(n) = 10 + 8n.$$

n	F_n	original ticks	optimized ticks
0	0	17	10
1	1	29	18
2	1	41	26
5	5	77	50
10	55	137	90
20	6,765	257	170
50	12,586,269,025	617	410
92	7,540,113,804,746,346,429	1,121	746

The general optimization lesson: first minimize the useful instruction sequence under the room's register contract; then place those instructions so the man reaches them with the fewest arrows and blanks; finally pack and reroute whole rooms. Your Fibonacci improvement wins at the second level while keeping the outer layout unchanged.

If you remember only one sentence, remember this:

A holds the answer for the current index, B holds the answer for the next index, BP counts how many index advances remain, and one geometric lap performs exactly one advance.